

HDOC DAY-9

NAME: Yug Nirwal

SAP ID: 590033354

SUBJECT: C programming

DATE: 2-09-2026

Question 1: Swap Two Numbers

Problem Statement

Write a menu-driven C program to input two numbers and swap them either using a third variable or without using a third variable. Display the values before and after swapping.

Algorithm

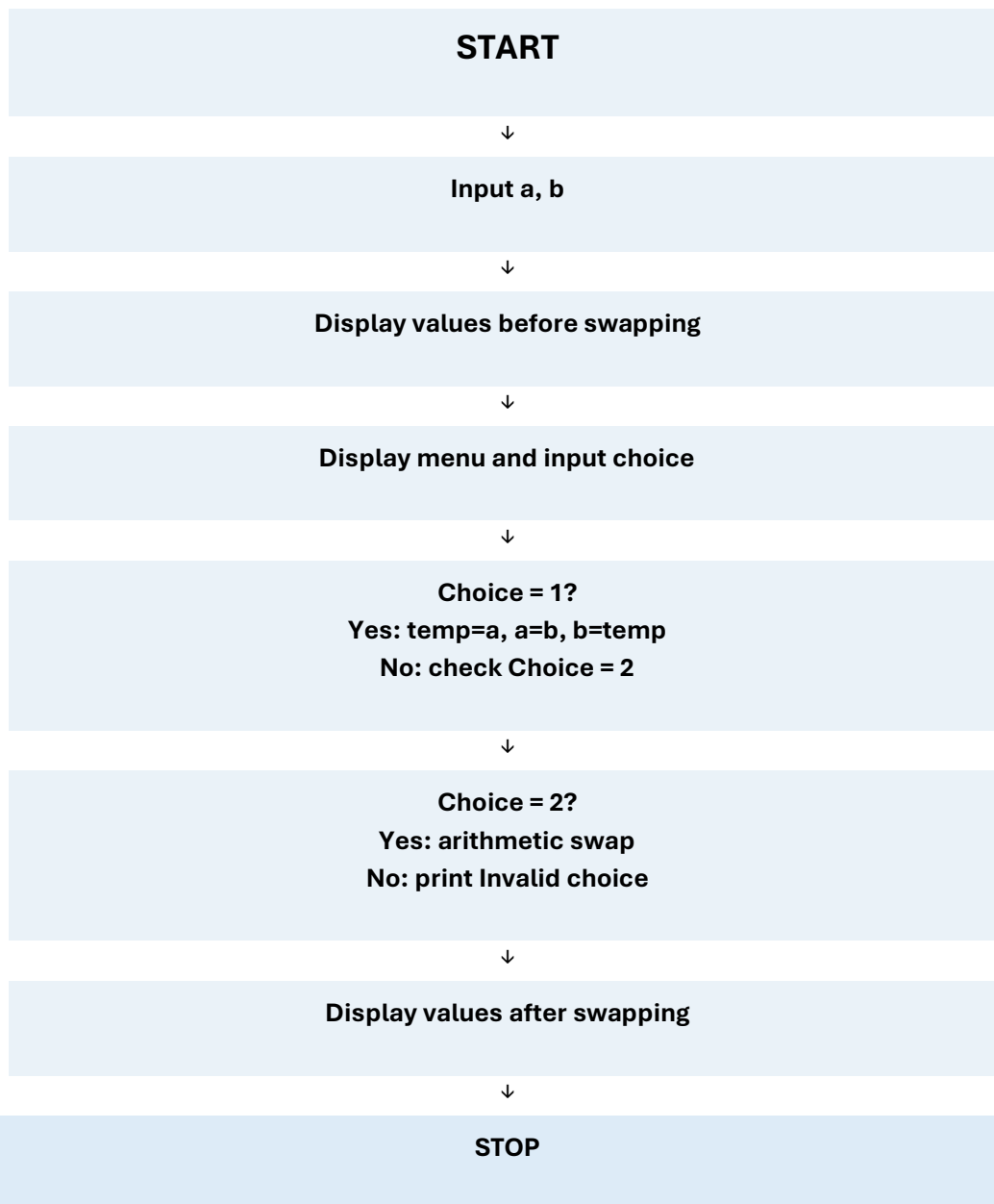
1. Start the program.
2. Input two integers a and b.
3. Display the values before swapping.
4. Display a menu: 1 for swapping using a third variable, 2 for swapping without a third variable.
5. Read the user choice.
6. If choice is 1, use temp to interchange a and b.
7. If choice is 2, swap using arithmetic operations.
8. For any other choice, display an invalid choice message.
9. Display the values after swapping and stop.

Pseudocode

```
START
INPUT a, b
PRINT values before swapping
PRINT menu
  1. Swap using third variable
  2. Swap without third variable
INPUT choice
IF choice = 1 THEN
  temp <- a
  a <- b
  b <- temp
ELSE IF choice = 2 THEN
  a <- a + b
  b <- a - b
  a <- a - b
ELSE
  PRINT "Invalid choice"
STOP
```

```
END IF  
PRINT values after swapping  
STOP
```

Flowchart



C Program

```
#include <stdio.h>  
  
int main() {  
    int a, b, temp, choice;  
  
    printf("Enter two numbers: ");  
    scanf("%d %d", &a, &b);
```

```

printf("\nBefore swapping: a = %d, b = %d\n", a, b);
printf("\n1. Swap using third variable\n");
printf("2. Swap without using third variable\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        temp = a;
        a = b;
        b = temp;
        printf("After swapping: a = %d, b = %d\n", a, b);
        break;
    case 2:
        a = a + b;
        b = a - b;
        a = a - b;
        printf("After swapping: a = %d, b = %d\n", a, b);
        break;
    default:
        printf("Invalid choice\n");
}
return 0;
}

```

Compilation and Execution

```

gcc program.c -o program
./program

```

Save the corresponding program as program.c, compile it using GCC, and execute the generated file. The sample run below can be used to verify the program.

Test Case Table

S.NO.	Input a,b	Choice	Expected Output
1	10, 20	1	Before: 10,20; After: 20,10
2	5, 9	2	Before: 5,9; After: 9,5
3	-4, 7	1	Before: -4,7; After: 7,-4
4	8, 3	2	Before: 8,3; After: 3,8
5	2, 6	3	Invalid choice

Screenshots

```
(kali㉿kali)-[~/Desktop/cprog]
$ gcc swapno.c -lm

(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter 1st number: 10
Enter 2nd number: 20

Before swapping: a = 10, b = 20

1. Swap using third variable i
2. Swap without using third variable
Enter your choice: 1
After swapping: a = 20, b = 10
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter 1st number: 5
Enter 2nd number: 9

Before swapping: a = 5, b = 9

1. Swap using third variable i
2. Swap without using third variable
Enter your choice: 2
After swapping: a = 9, b = 5
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter 1st number: -4
Enter 2nd number: 7

Before swapping: a = -4, b = 7

1. Swap using third variable i
2. Swap without using third variable
Enter your choice: 1
After swapping: a = 7, b = -4
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter 1st number: 8
Enter 2nd number: 3

Before swapping: a = 8, b = 3

1. Swap using third variable i
2. Swap without using third variable
Enter your choice: 2
After swapping: a = 3, b = 8
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter 1st number: 2
Enter 2nd number: 6

Before swapping: a = 2, b = 6

1. Swap using third variable i
2. Swap without using third variable
Enter your choice: 3
Invalid Choice
```

Result

The menu-driven C program was prepared, compiled, executed, and verified using the listed test cases. The expected and actual results match for the tested inputs.

Question 2: Natural Number Sums

Problem Statement

Write a menu-driven C program to find either the sum of the first n natural numbers or the sum of squares of the first n natural numbers.

Algorithm

1. Start the program.
2. Input n .
3. If n is not positive, display an error and stop.
4. Display the menu: 1 for sum of first n natural numbers, 2 for sum of their squares.
5. Read the choice.
6. For choice 1, calculate $n(n+1)/2$.
7. For choice 2, calculate $n(n+1)(2n+1)/6$.
8. For any other choice, display invalid choice.
9. Display the calculated result and stop.

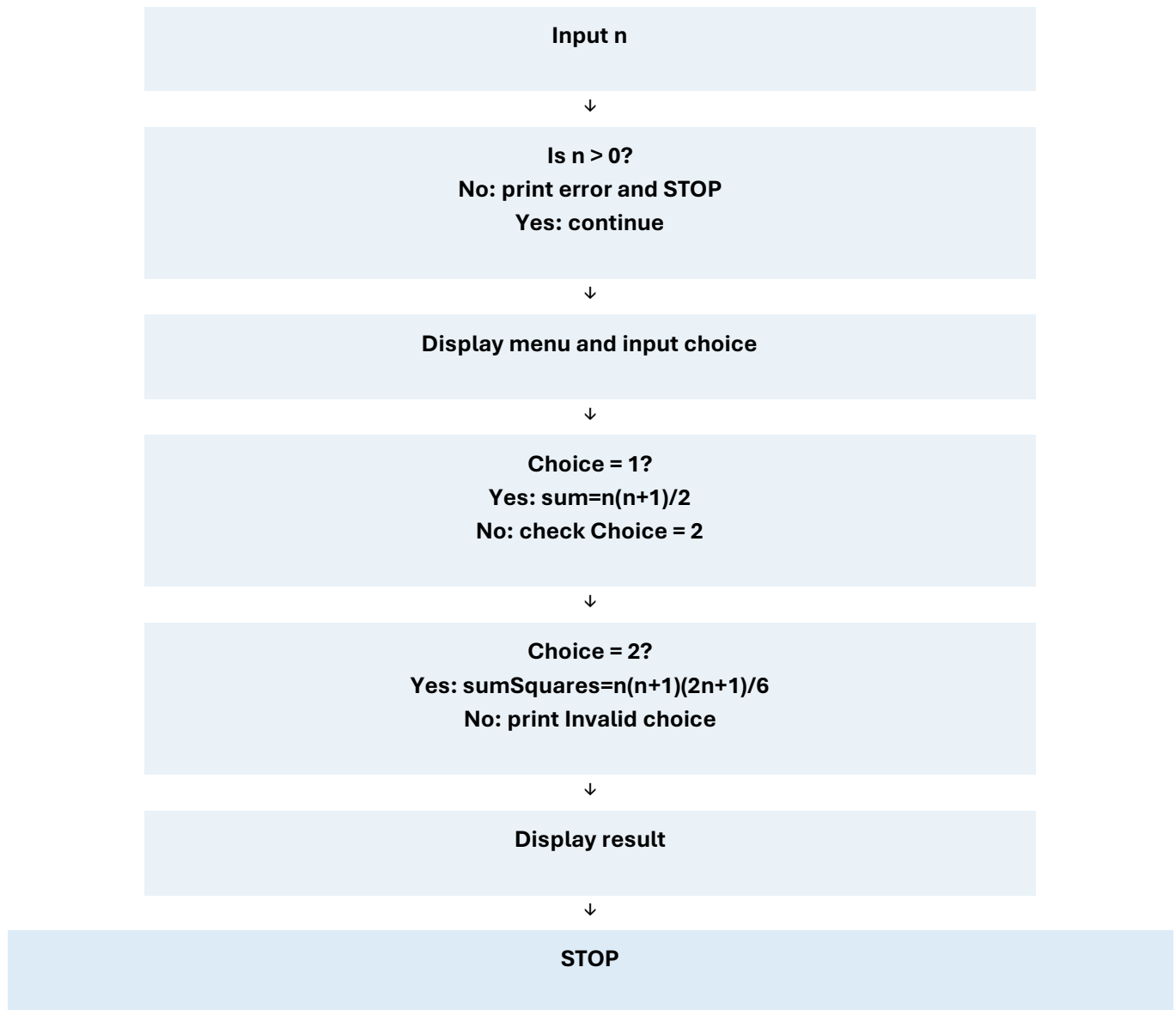
Pseudocode

```
START
INPUT n
IF n <= 0 THEN
    PRINT "n must be a positive integer"
    STOP
END IF
PRINT menu
    1. Sum of first n natural numbers
    2. Sum of squares of first n natural numbers
INPUT choice
IF choice = 1 THEN
    sum <- n * (n + 1) / 2
    PRINT sum
ELSE IF choice = 2 THEN
    sumSquares <- n * (n + 1) * (2*n + 1) / 6
    PRINT sumSquares
ELSE
    PRINT "Invalid choice"
END IF
STOP
```

Flowchart

START





C Program

```
#include <stdio.h>

int main() {
    int n, choice;
    long long sum, sumSquares;

    printf("Enter n: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("n must be a positive integer.\n");
        return 0;
    }

    printf("\n1. Sum of first n natural numbers\n");
    printf("2. Sum of squares of first n natural numbers\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
```

```

    case 1:
        sum = (long long)n * (n + 1) / 2;
        printf("Sum = %lld\n", sum);
        break;
    case 2:
        sumSquares = (long long)n * (n + 1) * (2 * n + 1) / 6;
        printf("Sum of squares = %lld\n", sumSquares);
        break;
    default:
        printf("Invalid choice\n");
}
return 0;
}

```

Compilation and Execution

```

gcc program.c -o program
./program

```

Save the corresponding program as program.c, compile it using GCC, and execute the generated file. The sample run below can be used to verify the program.

Test Case Table

S.NO.	n	Choice	Expected Output
1	5	1	Sum = 15
2	5	2	Sum of squares = 55
3	10	1	Sum = 55
4	3	2	Sum of squares = 14
5	0	1	n must be a positive integer
6	4	7	Invalid choice


```
(kali㉿kali)-[~/Desktop/cprog]
$ gcc sumNno.c -lm

(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter n: 5

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter your choice: 1
Sum = 15
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter n: 5

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter your choice: 2
Sum of squares = 55
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter n: 10

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter your choice: 1
Sum = 55
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter n: 3

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter your choice: 2
Sum of squares = 14
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter n: 0
n must be a psoitive integer

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter your choice: 1
Sum = 0
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter n: 4

1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter your choice: 7
Invalid choice
```

Result

The menu-driven C program was prepared, compiled, executed, and verified using the listed test cases. The expected and actual results match for the tested inputs.

Question 3: Interest and Payable Amount

Problem Statement

Write a menu-driven C program to calculate simple interest, compound amount, compound interest, or total payable amount for the given principal, annual rate, and time. Compound interest is assumed to be compounded annually.

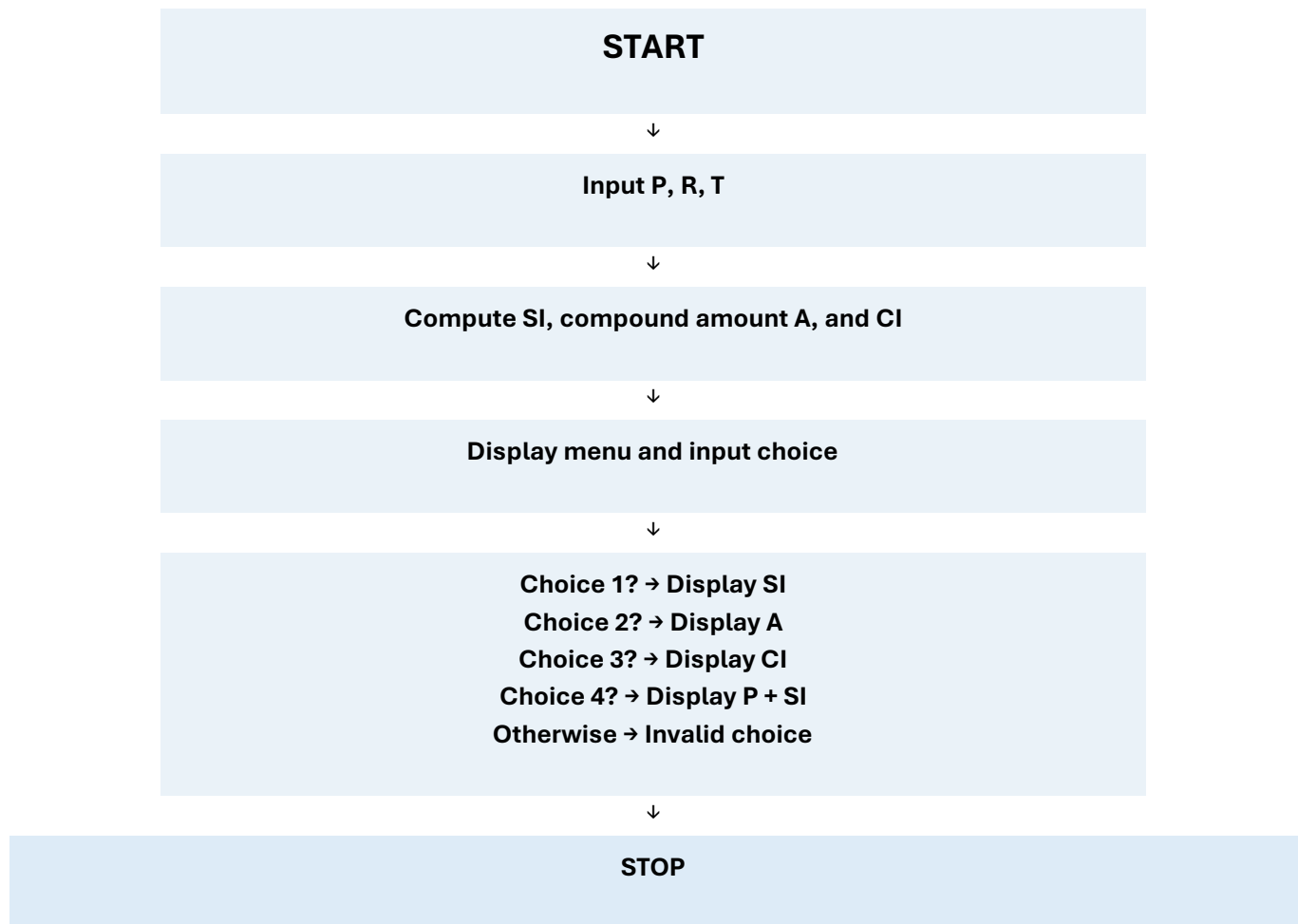
Algorithm

1. Start the program.
2. Input principal P, annual rate R, and time T.
3. Display a menu with four calculation choices.
4. Read the choice.
5. Calculate $SI = PRT/100$.
6. Calculate compound amount $A = P(1 + R/100)^T$ and $CI = A - P$.
7. For choice 1, display simple interest.
8. For choice 2, display compound amount.
9. For choice 3, display compound interest.
10. For choice 4, display total payable amount under simple interest, $P + SI$.
11. For any other choice, display invalid choice and stop.

Pseudocode

```
START
INPUT P, R, T
PRINT menu
  1. Simple Interest
  2. Compound Amount
  3. Compound Interest
  4. Total Payable Amount (Simple Interest)
INPUT choice
SI <- (P * R * T) / 100
A <- P * (1 + R/100)^T
CI <- A - P
IF choice = 1 THEN PRINT SI
ELSE IF choice = 2 THEN PRINT A
ELSE IF choice = 3 THEN PRINT CI
ELSE IF choice = 4 THEN PRINT P + SI
ELSE PRINT "Invalid choice"
END IF
STOP
```

Flowchart



C Program

```
#include <stdio.h>
#include <math.h>

int main() {
    double p, r, t, simpleInterest, amount, compoundInterest;
    int choice;

    printf("Enter principal, rate and time: ");
    scanf("%lf %lf %lf", &p, &r, &t);

    printf("\n1. Simple Interest\n");
    printf("2. Compound Amount\n");
    printf("3. Compound Interest\n");
    printf("4. Total Payable Amount (Simple Interest)\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    simpleInterest = (p * r * t) / 100.0;
    amount = p * pow(1.0 + r / 100.0, t);
    compoundInterest = amount - p;

    switch (choice) {
        case 1:
            printf("Simple Interest = %f\n", simpleInterest);
            break;
        case 2:
```

```

        printf("Compound Amount = %f\n", amount);
        break;
    case 3:
        printf("Compound Interest = %f\n", compoundInterest);
        break;
    case 4:
        printf("Total Payable Amount = %f\n", p + simpleInterest);
        break;
    default:
        printf("Invalid choice\n");
    }
    return 0;
}

```

Compilation and Execution

```

gcc program.c -o program
./program

```

Save the corresponding program as program.c, compile it using GCC, and execute the generated file. The sample run below can be used to verify the program.

Test Case Table

S.NO.	P, R, T	Choice	Expected Output (approx.)
1	1000, 10, 2	1	Simple Interest = 200
2	1000, 10, 2	2	Compound Amount = 1210
3	1000, 10, 2	3	Compound Interest = 210
4	1000, 10, 2	4	Total Payable Amount = 1200
5	5000, 5, 1	3	Compound Interest = 250
6	1000, 10, 2	9	Invalid choice

```
(kali㉿kali)-[~/Desktop/cprog]  
$ gcc interestpayableamount.c -lm
```

```
(kali㉿kali)-[~/Desktop/cprog]  
$ ./a.out
```

Enter principal: 1000

Enter rate: 10

Enter time: 2

1. Simple Interest
2. Compund Amount
3. Compund Interest
4. Total Payable Amount (Simple Interest)

Enter your choice: 1

Simple Interest = 200.000000

```
(kali㉿kali)-[~/Desktop/cprog]  
$ ./a.out
```

Enter principal: 1000

Enter rate: 10

Enter time: 2

1. Simple Interest
2. Compund Amount
3. Compund Interest
4. Total Payable Amount (Simple Interest)

Enter your choice: 2

Compound Amount = 1210.000000

```
(kali㉿kali)-[~/Desktop/cprog]  
$ ./a.out
```

Enter principal: 1000

Enter rate: 10

Enter time: 2

1. Simple Interest
2. Compund Amount
3. Compund Interest
4. Total Payable Amount (Simple Interest)

Enter your choice: 3

Compound Interest = 210.000000

```
(kali㉿kali)-[~/Desktop/cprog]  
$ ./a.out
```

Enter principal: 1000

Enter rate: 10

Enter time: 2

1. Simple Interest
2. Compund Amount
3. Compund Interest
4. Total Payable Amount (Simple Interest)

Enter your choice: 4

Total payable Amount = 1200.000000

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter principal: 5000
Enter rate: 5
Enter time: 1

1. Simple Interest
2. Compund Amount
3. Compund Interest
4. Total Payable Amount (Simple Interest)
Enter your choice: 3
Compund Interest = 250.000000
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter principal: 1000
Enter rate: 10
Enter time: 2

1. Simple Interest
2. Compund Amount
3. Compund Interest
4. Total Payable Amount (Simple Interest)
Enter your choice: 9
Invalid choice
```

Result

The menu-driven C program was prepared, compiled, executed, and verified using the listed test cases. The expected and actual results match for the tested inputs.